

Advanced Virtual Mouse Gesture and Voice Control System

PROJECT REPORT

Submitted by

ADIL MOHAMMED SHAJAHAN SBC20CS002

AKHIL NORMAN SBC20CS004

JAIS JOHN SBC20CS068



To

*The APJ Abdul Kalam Technological University in partial
fulfillment of the requirements for the award of the degree of*

BACHELOR OF TECHNOLOGY IN
COMPUTER SCIENCE AND ENGINEERING



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

SREE BUDDHA COLLEGE OF ENGINEERING

ALAPPUZHA - 690 529

MAY 2024

DECLARATION

We undersigned hereby declare that the project report **Advanced Virtual Mouse Gesture and Voice Control System**, submitted for partial fulfillment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala is a bonafide work done by us under the supervision of Ms. Reshmi S. (Asst.Prof, Dept of CSE). This submission represents our ideas in our own words and where ideas or words of others have been included, we have adequately and accurately cited and referenced the original sources. We also declare that we have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in my/our submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

Place : Pattoor

Date : 01-05-2024

ADIL MOHAMMED SHAJAHAN
AKHIL NORMAN
JAIS JOHN

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING,
SREE BUDDHA COLLEGE OF ENGINEERING
PATTOOR P.O., ALAPPUZHA**



CERTIFICATE

This is to certify that the report entitled “**Advanced Virtual Mouse Gesture and Voice Control System**” submitted by **Mr. Adil Mohammed Shajahan(SBC20CS002)**, **Mr. Akhil Norman(SBC20CS004)**, **Mr. Jais John(SBC20CS068)** to the APJ Abdul Kalam Technological University in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering is a bonafide record of the project work carried out by them under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Ms. Reshmi S	Dr. Anju J Prakash	Dr. S.V. Annlin Jeba
Internal Supervisor	Project Coordinator	Head of the Department
(Assistant Professor)	(Associate Professor)	(Head of the department)
Dept. of CSE	Dept. of CSE	Dept. of CSE

ACKNOWLEDGEMENT

We express our sincere gratitude to our respected principal **Dr. K Krishnakumar**, for his valuable support and advice.

We express our sincere thanks to **Dr. S.V. Annlin Jeba**, Head of the Department, Department of Computer Science and Engineering, Sree Buddha College of Engineering Pattoor, for her valuable guidance and suggestions throughout the entire project work.

We would like to thank our project coordinator, **Dr. Anju J Prakash**, Associate.Prof, Department of Computer Science and Engineering, Sree Buddha College of Engineering, Pattoor for the guidance and advice during the work.

We would like to thank with a deep sense of gratitude and obligation to our project supervisor, **Ms. Reshmi S**, Asst.Prof, Department of Computer Science and Engineering, Sree Buddha College of Engineering, Pattoor for her valuable guidance and suggestions throughout the entire project work.

We express our sincere gratitude to all the members of the Department of Computer Science and Engineering, Sree Buddha College of Engineering, for their encouragement and valuable assistance.

In particular, we are thankful to all those who have helped me directly or in- directly in completing this work.

Above all we thank Almighty for giving me the health and strength to complete the work on time.

ADIL MOHAMMED SHAJAHAN

AKHIL NORMAN

JAIS JOHN

ABSTRACT

A cutting-edge virtual mouse control system that incorporates voice commands, gesture detection, and eye tracking features to transform human-computer interaction. This complex system improves productivity and accessibility for a larger group of users, including people with various physical and visual disabilities. With the use of sophisticated computer vision algorithms, users may utilize natural hand gestures to manipulate the pointer, click, and carry out other standard mouse operations in a virtual world thanks to the gesture recognition capability. By enabling users to move the pointer and maybe execute actions based on where they gaze on the screen, eye tracking provides an additional level of control. Since individuals may now communicate with the computer interface without using conventional mice, this proves to be very helpful for those who have trouble moving about or with dexterity. With natural language processing, voice commands enable hands-free window and application navigation for users. This increases the system's customization capabilities. Its all-inclusive method serves people with ailments such as carpal tunnel syndrome, rheumatoid arthritis, upper limb impairments, or neuromuscular illnesses, for which conventional mice are problematic. In general, the Virtual Mouse Control System aims to improve everyone's computing experience by addressing accessibility issues and making technology more approachable for all. Through the smooth integration of gesture detection, eye tracking, and voice commands, the system opens the door to a more flexible and intuitive paradigm for human-computer interaction, removing obstacles and enabling a greater variety of people to engage with computers more successfully.

Example: MediaPipe, Machine Learning, Gesture Recognition, Virtual Mouse

CONTENTS

ACKNOWLEDGEMENT	i
ABSTRACT	ii
LIST OF FIGURES	vi
ABBREVIATIONS	viii
CHAPTER 1 INTRODUCTION	1
1.1 Overview	1
1.2 Objective	2
CHAPTER 2 LITERATURE SURVEY	3
2.1 A Real Time Hand Gesture Recognition System using Motion History Image	3
2.2 Gesture Recognition with a Data Glove	3
2.3 Virtual Mouse Control using Colored Fingertips and Hand Gesture Recognition	5
2.4 RADAR based Object Detector using Ultrasonic Sensor	6
2.5 Human-Eye Controlled Virtual Mouse	7
2.6 Voice and Gesture based Virtual Desktop Assistant for Physically Disabled	8
CHAPTER 3 METHODOLOGY	9
3.1 System Overview	9
3.1.1 OpenCV	9
3.1.2 MediaPipe	10
3.1.3 Speech Recognition	11
3.1.4 PyAuto GUI	12
3.1.5 TKinter Library	12
3.1.6 Virtual Mouse System's Camera and Microphone	13

3.1.7 Video Recording and Analyzing	14
3.1.8 Virtual Screen Matching	14
3.1.9 Recognising the Finger which is up and Performing Out the appropriate Mouse Operation	15
3.1.10 Recognising the Commands used for Different Modes of Mouse Operation	16
3.2 System Architecture	16
3.2.1 GUI	16
3.2.2 Gesture Mode	18
3.2.2.1 Image Capture	18
3.2.2.2 Hand Detection	19
3.2.2.3 Gesture Recognition	20
3.2.2.4 Mouse Cursor Control	21
3.2.3 Gesture & Voice Mode	22
3.2.4 Eye Mode	23
CHAPTER 4 RESULT AND DISCUSSION	25
4.1 Experimental Setup	25
4.2 Screenshots	25
4.2.1 Hand Tracking	25
4.2.2 GUI	26
4.2.3 Gesture Mode	26
4.2.4 Gesture & Voice Mode	27
4.2.5 Eye Mode	28

CHAPTER 5 CONCLUSION	29
CHAPTER 6 FUTURE SCOPE	30
REFERENCES	31

LIST OF FIGURES

FIGURE NO:	TITLE	PAGE NO:
3.1	OpenCV Block Diagram	10
3.2	MediaPipe Hand Recognition	11
3.3	Speech Recognition	12
3.4	TKinter Window	13
3.5	Custom TKinter Window	13
3.6	Co-ordinates in Hand	14
3.7	Mouse Functions	15
3.8	Architecture of accessing Different Modes of Operation	17
3.9	Flow Chart of Gesture Mode	18
3.10	Flow Chart of Eye Mode	24
4.1	Hand Tracking	25
4.2	Mode Selector	26
4.3	Neutral Action	26
4.4	Cursor Move	26
4.5	Left Click	26
4.6	Right Click	26
4.7	Double Click	27

4.8	Scrolling, Brightness and Volume Control	27
4.9	Drag and Drop	27
4.10	Left Click Command Execution	27
4.11	Right Click Command Execution	28
4.12	Cursor Movement using Eye	28

ABBREVIATIONS

HCI	Human-Computer Interaction
ML	Machine Learning
AI	Artificial Intelligence
SVM	Support Vector Machines
UI	User Interface
GUI	Graphical User Interface
CTk	Custom TKinter
HMM	Hidden Markov Model

CHAPTER 1

INTRODUCTION

1.1 Overview

At the forefront of human-computer interaction (HCI), the Virtual Mouse Gesture Control System with Integrated Voice Commands and Eye Tracking expands the boundaries of user experience. This innovative system addresses the limitations of traditional input devices like keyboards and mice, which have long dominated computer interaction. Recognizing the need for alternative methods, this project strives to create a comprehensive and user-friendly window navigation system by merging the intuitive nature of gestures, the efficiency of voice commands, and the precision of eye tracking. Gesture Recognition Technology leverages computer vision algorithms to interpret and respond to hand movements in real-time. Extending this development, the Virtual Mouse Gesture Control System enables users to manipulate clicks, cursor movements, and other standard mouse operations in a virtual environment by employing their natural hand gestures. A more organic and engaging user experience is promoted by this hands-free method, which is especially helpful in scenarios where conventional input devices could be difficult to use or inconvenient. By using voice commands, The Power of Voice Commands improves usability even further. With the use of natural language processing, users can voice commands to their computer directly and engage with their digital environment. This enables voice commands to open, close, and switch between programs, allowing users to operate windows and apps hands-free. The interface can now accommodate a wider range of user wants and preferences thanks to the new control layer's flexibility and adaptability. With the use of the Eye Tracking technology, users can move the cursor and even carry out tasks by simply looking at the screen. This is especially helpful for people who have trouble moving about or using their dexterity because it provides an alternate mode of interaction that does not require any physical input devices at all. By serving a larger user base, the Virtual Mouse Gesture Control System puts accessibility first. The gesture recognition and eye tracking features are beneficial for those with illnesses such as carpal tunnel syndrome, arthritis, upper limb limitations, or neuromuscular diseases since they offer substitute engagement options without the need of traditional mouse. A more flexible and intuitive HCI paradigm is made possible by the Virtual Mouse Gesture Control System, which seamlessly integrates gesture detection, eye tracking, and voice instructions.

1.2 Objective

The overarching objective of this project is to design and develop a comprehensive Virtual Mouse Gesture Control System that revolutionizes human-computer interaction (HCI) by offering a multifaceted approach to navigating the digital world. This system transcends the limitations of traditional mice and keyboards by integrating three key functionalities:

Gesture Recognition: Utilizing computer vision algorithms, the system will recognize and interpret hand gestures in real-time, allowing users to control cursor movement, perform clicks, and execute other conventional mouse actions within a virtual environment.

Eye Tracking: This feature will add another layer of control, enabling users to move the cursor and potentially perform actions based on where they look on the screen. Eye tracking will be particularly beneficial for individuals with mobility limitations or dexterity issues.

Gesture with Voice Commands: By leveraging natural language processing capabilities, the system will empower users to navigate windows and applications hands-free through voice commands. This will encompass functionalities like opening, closing, and switching between programs where extra gestures is not needed.

This multifaceted approach aims to achieve the following specific goals:

Transform Cursor Control: Through intuitive, hands-free methods of gesture recognition and eye tracking, the project seeks to completely revolutionize the way users interact with the cursor. This approach will overcome the limitations and potential discomfort associated with conventional mice and keyboards, especially for users with Limited Mobility, Desire for a More Seamless Interface, Enhanced Accessibility, Visual Impairments, Simplified User Experience.

The project leverages pre-existing computer vision libraries (e.g., MediaPipe, OpenCV) and speech recognition tools to ensure compatibility with a variety of user setups and minimize the need for specialized hardware or software installations. This approach fosters greater accessibility and broadens the system's reach to a larger audience.

By fusing voice commands and hand gestures with eye tracking functionalities, the project prioritizes natural and intuitive interaction. This user-centric approach aims to bridge the gap between human behavior and computer interaction, resulting in a more seamless, engaging, and enjoyable user experience.

CHAPTER 2

LITERATURE SURVEY

2.1 D.H. Liou, D. Lee and C.C. Hsieh, “A Real Time Hand Gesture Recognition System using Motion History Image” [2]

D.H. Liou, D. Lee, and C.C. "A real time hand gesture recognition system using motion history image" proposes a real-time hand gesture recognition system using motion history image (MHI). The system first detects hands in the image using an adaptive skin color model based on face detection. Then, it extracts features from the MHI to classify hand gestures. The system can recognize six hand gestures: up, down, left, right, fist, and waving. Experiments show that the system has an average accuracy of 94.1% and a processing time of 3.81 ms per frame.

Disadvantage:

- **Limited Gesture Set:** The system is currently only able to recognize six hand gestures, which may be insufficient for some applications.
- **Sensitivity to Background Noise:** The performance of the system may be affected by background noise, such as moving objects or complex backgrounds.
- **Variation in Hand Appearances:** The system's accuracy may vary depending on the individual's hand shape, skin color, and lighting conditions
- **Limited Testing Data:** The system was evaluated on a relatively small dataset of 250 hand gestures performed by five individuals. A larger and more diverse dataset would provide a more comprehensive assessment of the system's generalizability.
- **Limited Application to Real-World Scenarios:** The paper focuses on controlled laboratory settings, and further research is needed to evaluate the system's performance in real-world environments with varying lighting conditions, backgrounds, and user interactions.

2.2 D. L. Quam, “Gesture Recognition with a Data Glove” [1]

D. L. Quam's paper "Gesture recognition with a Data Glove" proposes a method for using a data glove to recognize hand gestures. The sensors typically measure the bending of the fingers and wrist, as well as the acceleration and orientation of the hand. Quam's method

works by first collecting data from the data glove while the user is performing a variety of hand gestures. The data is then used to train a machine learning algorithm to recognize the different gestures. Once the algorithm is trained, it can be used to recognize new gestures in real time. Here is a more detailed explanation of how Quam's method works:

Data collection: The first step is to collect data from the data glove while the user is performing a variety of hand gestures. The data is typically collected at a high frequency, such as 100 Hz, so that the movement of the hand can be captured accurately.

Data preprocessing: Once the data has been collected, it is pre-processed to remove any noise or artifacts. This may involve filtering the data, smoothing it, or normalizing it.

Feature extraction: The next step is to extract features from the preprocessed data. Features are characteristics of the data that can be used to distinguish between different gestures. Some common features for hand gesture recognition include the bending of the fingers and wrist, the acceleration of the hand, and the orientation of the hand.

Machine learning: The extracted features are then used to train a machine learning algorithm to recognize the different gestures. There are many different machine learning algorithms that can be used for hand gesture recognition, such as support vector machines (SVMs), decision trees, and neural networks.

Gesture recognition: Once the machine learning algorithm is trained, it can be used to recognize new gestures in real time. To do this, the algorithm is simply given the extracted features from the current frame of data, and it outputs a prediction of the gesture that is being performed.

Quam's method has been shown to be very effective for hand gesture recognition. In one study, Quam's method achieved an accuracy of 99.5% on a dataset of 250 hand gestures.

Disadvantage:

- Data Gloves are not well-suited for application that require continuous or unrestricted hand movement and also data gloves can be difficult to clean and sanitize
- Cost: Data glove are typically more expensive than other gesture recognition method

2.3 V. V. Reddy, T. Dhyanchand, G. V. Krishna and S. Maheshwaram, "Virtual Mouse Control Using Colored Finger Tips and Hand Gesture Recognition" [3]

V. V. Reddy, T. Dhyanchand, G. V. Krishna, and S. Maheshwaram "Virtual Mouse Control Using Coloured Finger Tips and Hand Gesture Recognition", propose a system for controlling a virtual mouse using coloured finger tips and hand gesture recognition. The system uses a webcam to capture images of the user's hand. It then uses colour detection to identify the coloured finger tips and hand gesture recognition to identify the gesture that the user is performing. The system uses two different methods for tracking the fingers:

- Coloured finger caps: The user wears coloured caps on their fingertips. The system uses colour detection to identify the coloured caps in the webcam image.
- Hand gesture detection: The system uses hand gesture recognition to detect the contour of the hand in the webcam image. It then forms a convex hull around the contour and extracts features from the convex hull, such as the area ratio of the contour and hull formed.

Once the fingers have been tracked, the system uses the position of the fingers to control the virtual mouse cursor. The system also uses hand gesture recognition to identify the gesture that the user is performing. The system can recognize a variety of gestures, such as moving the cursor, clicking the left and right mouse buttons, and scrolling the mouse wheel. The system was evaluated on a dataset of 20 hand gestures performed by 10 people. The system achieved an average accuracy of 95%..The system can be used in a variety of applications, such as:

- Human-computer interaction: The system can be used to control computers and other devices without the need for a physical mouse.
- Accessibility: The system can be used by people with disabilities to control their computers.
- Gaming: The system can be used to control games in a more immersive and engaging way.
- Education: The system can be used to create educational games and simulations that are interactive and engaging.

Disadvantages:

- The system performance may be affected by background clutter, such as moving objects or complex backgrounds. This could lead to misclassification or false positives

Overall, the system proposed by Reddy et al. is a promising new way to control computers and other devices using coloured finger tips and hand gesture recognition.

2.4 Akshaya U Kulkarni, Amit M Potdar “RADAR based Object Detector using Ultrasonic Sensor” [5]

An electronic gadget known as an ultrasonic sensor uses ultrasonic sound waves to measure an object's distance from it and then transforms the sound waves' reflection into electrical signals. These waves, which usually range in frequency from 20 kHz to 1 GHz, are above the range of human hearing. RADAR-based object detector using an ultrasonic sensor is a system designed to detect and locate objects in its vicinity by using a combination of radar technology and ultrasonic sensors. This kind of system is commonly used in various applications, including autonomous vehicles, robotics, and industrial automation, where accurate object detection and ranging are essential. Here's a breakdown of how such a system might work:

Radar Technology: RADAR stands for "Radio Detection and Ranging." It uses radio waves (typically in the microwave or millimeter-wave frequency range) to detect the presence and location of objects. RADAR emits electromagnetic waves and then listens for the reflected signals to determine the distance and velocity of objects. It can work in various modes, including continuous wave (CW) and pulse-Doppler.

Ultrasonic Sensor: Ultrasonic sensors, on the other hand, use sound waves with frequencies higher than the human audible range. These sensors emit high-frequency sound waves and listen for the echoes produced when these waves hit an object and bounce back. The time it takes for the sound waves to return to the sensor is used to calculate the distance to the object.

Integration: In a RADAR-based object detector using an ultrasonic sensor, both RADAR and ultrasonic sensors work together.

Here's how they can be integrated:

The RADAR system provides a broad view of the surroundings and can detect objects at longer ranges but with less accuracy in terms of precise distance measurement. The ultrasonic

sensor, on the other hand, provides accurate distance measurements at a shorter range but may have limitations in terms of field of view and performance in adverse conditions like heavy rain or fog.

Data Fusion: To create a more robust and accurate object detection system, the data from both the RADAR and ultrasonic sensor can be fused. This means combining the information from both sensors to get a more complete picture of the environment.

Applications: RADAR-based object detectors using ultrasonic sensors are used in various applications, such as collision avoidance in vehicles, parking assistance systems, industrial automation for object tracking and safety, and robotics for navigation and obstacle avoidance. In summary, a RADAR-based object detector using an ultrasonic sensor combines the advantages of both RADAR and ultrasonic technologies to provide accurate and reliable object detection and ranging. The integration of these two sensors and the fusion of their data enable the system to operate effectively in a wide range of environments and applications.

Disadvantages:

Ultrasonic sensors have a limited range and also not accurate as radar sensors.

2.5 Mr. Dhanaraju, Dr. Sreenivas Mekala, A. Harsha Vardhan Rao, CH. Pavan Kumar, R. Lokesh “Human-Eye Controlled Virtual Mouse” [7]

The goal of the proposed system is to facilitate the work of "amputees," or individuals who are unable to move their arms. Specifically, the system will enable individuals who are paralyzed in all four limbs, or amputees, to use and operate a mouse by using their facial features and eye movements. In order to create a user-friendly human-computer interface, the system will only need a camera to use an individual's eyes and facial characteristics as a pointing device for the computer system. The research uses a cursor mapping system to identify and record facial features in video. The program needs to extract every frame from the video when the camera is opened. Every frame of the video will be processed in roughly 1/30th of a second because the frame rate is usually around 30 frames per second. After that, the application goes through a number of stages to determine and map the video's attributes to the main points. The facial regions need to be recognized once the frame has been recovered. The frames will therefore undergo a series of image-processing operations to properly analyze the frame, enabling the algorithm to discern features like lips, noses, and

eyes. The system performs the action like opening the mouth, eye wink, squinting eye and head movement. Python language, SVM and HOG are used.

Cursor movement is done by pointing the users nose towards the left, right, top and bottom of a rectangle. For scrolling, the user needs to squint the eyes and like cursor movement move the nose inside the rectangle to scroll to that desired area.

Disadvantages:

Blinking of eye is an involuntary action, continuously use blinking as the mode of mouse operation it may lead to discomfort to the users eye and in future it causes head ache.

2.6 Tata Jagannadha Swamy, M Nandini, Nandini B, Venkata Karthika K, V Laxmi Anvitha and Ch Sunitha “Voice and Gesture based Virtual Desktop Assistant for Physically Challenged People” [11]

This proposes that via voice and gesture detection, the Voice and Gesture based Virtual Desktop Assistant (VGODA) technology enables computer interaction for those with physical disabilities. It is compatible with the majority of voice-activated virtual assistants and is meant to be used on desktop computers. With the use of a webcam and a microphone, VGODA may receive gesture and spoken input. Text is generated from the audio input, and a trained model is used to identify the movements. Next, from the text and motions, the corresponding commands are retrieved, and the jobs are started. Either the helper reads aloud or displays the output. Compared to standard virtual assistants, which usually only accept voice input, VGODA provides a number of advantages. First off, those with physical disabilities who might have trouble communicating can access it more easily. It can be utilized for a greater variety of jobs, making it more versatile in the second place. Thirdly, because it can detect gestures instantly, it is more effective.

CHAPTER 3

METHODOLOGY

3.1 System Overview

For hand tracking, gesture recognition and eye tracking the MediaPipe framework is employed, for computer vision we use the OpenCV library. The technique tracks, recognizes hand gestures and finger tips using ML concepts. For voice commands, Speech Recognition library is used as it converts the voice to text by recognizing the language or words that a person speaks later that text is used as commands for further operations.

3.1.1 OpenCV

For images, OpenCV offers object detection techniques. Using the OpenCV module, the best applications of computer vision can be developed. This library is utilized for face and object identification, picture and video data processing, and video analysis.

OpenCV provides a wide range of tools and functions for working with images and videos. Some of the key features and capabilities of OpenCV include:

Image Processing: OpenCV offers a variety of image processing functions, such as filtering, edge detection, image enhancement, and more.

Object Detection: OpenCV includes tools for object detection, face recognition, and object tracking. It supports popular object detection frameworks like Haar cascades and deep learning models.

Feature Detection and Matching: OpenCV provides algorithms for key point detection, feature matching, and image stitching, which are crucial for applications like panorama creation.

Machine Learning: OpenCV has a machine learning module that supports various machine learning techniques, including decision trees, k-nearest neighbors, support vector machines, and more.

Camera Calibration: It helps calibrate cameras, correct for lens distortions, and perform 3D reconstruction from multiple images.

Video Analysis: OpenCV enables real-time video analysis, including motion tracking, object tracking, and video stabilization.

3D Computer Vision: It supports 3D scene reconstruction and point cloud processing from stereo cameras or depth sensors.

Graphical User Interface (GUI): OpenCV includes GUI features to create interactive applications and display images and videos in windows.

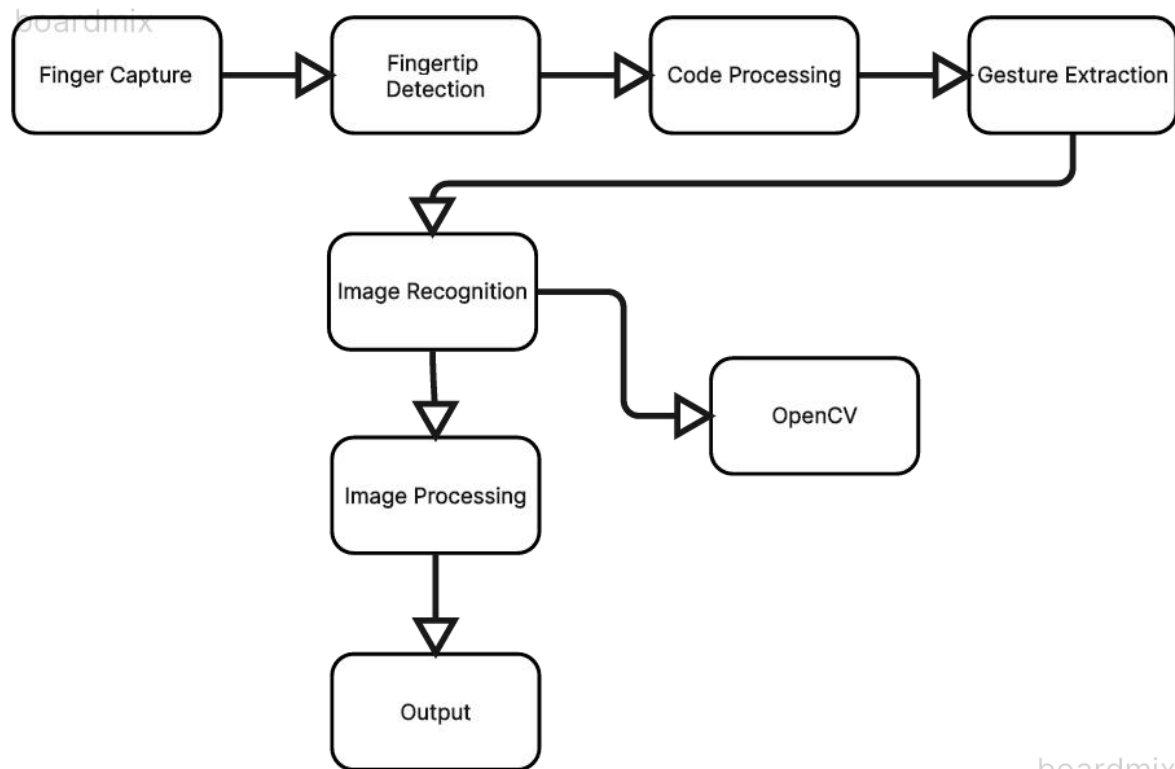


Fig 3.1: OpenCV Block Diagram

3.1.2 MediaPipe

A machine learning pipeline employs a Google open-source framework known as MediaPipe. The MediaPipe framework may be employed for cross-platform computing as it was developed leveraging time series data. The MediaPipe architecture supports multiple audio and video formats since it is multimodal. The developer uses MediaPipe framework to build systems for application- related objectives as well as to design and analyze systems that employ graphs.

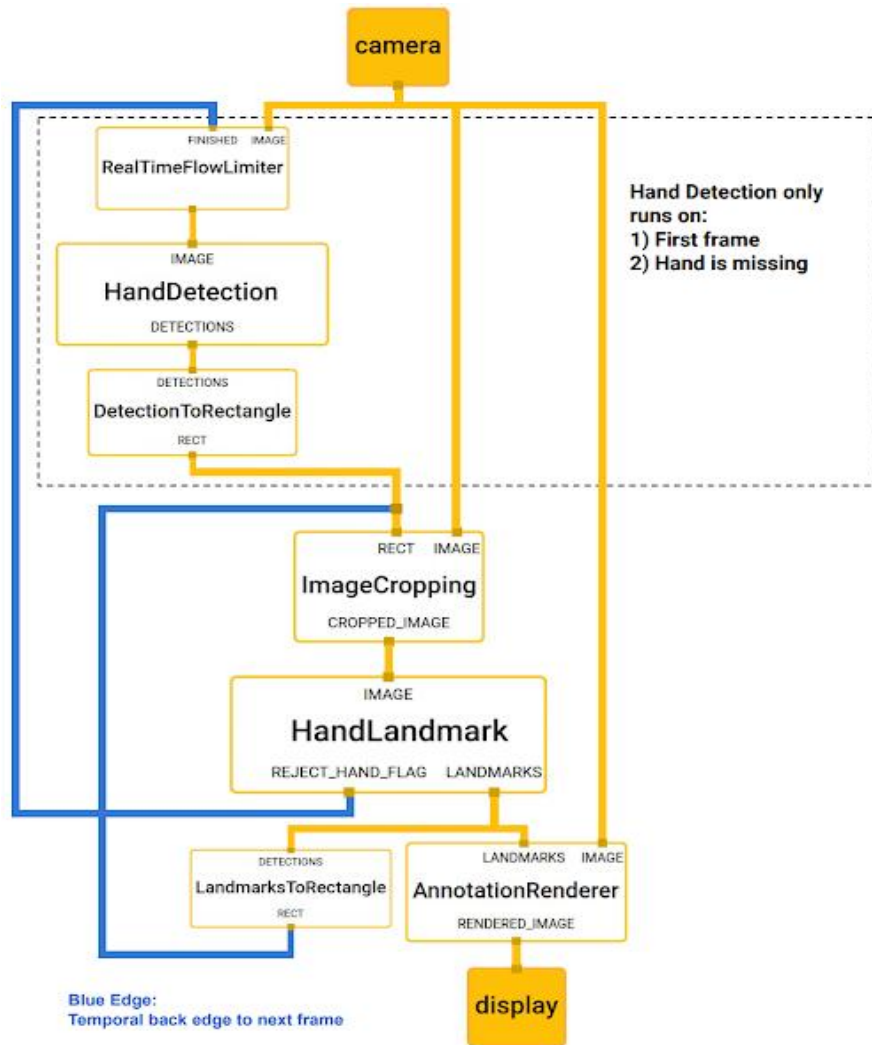


Fig 3.2: MediaPipe Hand recognition

3.1.3 Speech Recognition

Transcribing audible words into text is known as speech recognition. This technology allows computers, other machines, or other devices to comprehend, translate, and transcribe spoken language into a legible format.

First, the uttered words are translated into text using a technique called automated speech recognition (ASR), sometimes referred to as voice-to-text conversion. The spoken commands are recorded by the system through audio input. To accurately depict the speech content, a number of characteristics from the audio signal are taken out. Statistical models predict the order of words or phrases based on the likelihoods of word pairings in a given language and link the extracted audio properties to phonemes, or fundamental units of sound, in that

language. It then creates a word sequence that most closely resembles the spoken input by matching the extracted attributes with the linguistic and acoustic models. After the spoken words are translated into text, the system must comprehend the meaning and intention of the original language. For this, methods from Natural Language Understanding (NLU) are applied. The text is interpreted as a command by the system once it has been understood. This entails assigning particular actions or system functionalities to the identified text. For instance, if the text that the system recognizes is "Set a reminder for 5 PM," it will interpret it as an instruction to set a reminder for that time. The requested action is then carried out by the system as it executes the command. Depending on the nature of the command and the capabilities of the system, this could entail actions like sending a message, scheduling a reminder, starting a search, managing devices, or performing any other activity.

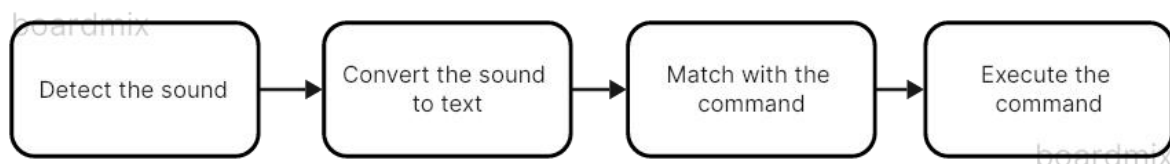


Fig 3.3: Speech Recognition

3.1.4 PyAuto GUI

PyAutoGUI is a Python library that lets the user to control the computer's mouse and keyboard. It can automate repetitive tasks, click buttons, type text, and more. It's easy to use and works on Windows, Mac, and Linux. Click buttons and menus, Type text , Drag and drop files, Scroll up and down, Take screenshots these are some functions which can be done by using PyAutoGUI. PyAutoGUI uses Python's built-in modules to interact with the computer's desktop environment. It can simulate mouse clicks and key presses, and it can also get information about the screen, such as the position of windows and the colors of pixels. Benefits of PyAutoGUI are Simple and easy to use, Cross-platform, Powerful.

3.1.5 TKinter Library

An easy-to-use interface for constructing Graphical User Interfaces (GUIs) for Python programs is offered by the Tkinter package. Most common Python distributions pre-install Tkinter, thus there's no need for additional installations. For developers who wish to quickly design GUIs without requiring further setup, this makes it a convenient option. Object-

oriented programming is the foundation of Tkinter. It offers a number of pre-built widgets (such as text fields, buttons, and labels) that act as the fundamental pieces of the graphical user interface. Tkinter's object-oriented design allows you to arrange and personalize these widgets. In general, Tkinter applications function flawlessly on a variety of operating systems, including Linux, macOS, and Windows. A comparatively easy GUI creation syntax is provided by Tkinter. Python code is used to define the appearance and functionality of the windows in the program.

Here, we are using extension of TKinter GUI instead of Tkinter that is Custom TKinter which is more modern looking than traditional TKinter dialog box. Custom TKinter (CTK) makes a promising look for the user to choose the modes of mouse operation at ease. Custom TKinter provides additional UI elements which includes components like CTkButton, CTkEntry, CTkProgressBar, etc. It offers built-in light and dark themes that you can easily switch between, or we can define custom color schemes to match the application's branding. Custom Tkinter provides a more intuitive way to handle user interactions with GUI elements, making the code cleaner and easier to maintain. Custom Tkinter ensures the GUIs display correctly on high-resolution monitors. In essence, CustomTkinter provides a layer of polish and additional functionalities on top of the standard Tkinter library. It allows developers to create more visually appealing, feature-rich, and user-friendly GUIs while still leveraging the core functionalities of Tkinter.

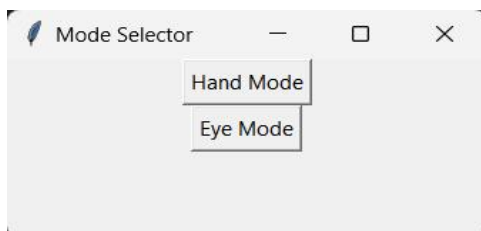


Fig 3.4: TKinter Window

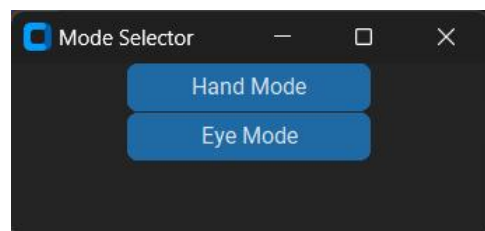


Fig 3.5: Custom TKinter Window

3.1.6 Virtual Mouse System's Camera and Microphone.

This AI virtual mouse system relies on the images from a laptop or PC. The video capture object is generated by using Python computer vision toolkit OpenCV and web camera used for recording. The web cam provides frames to virtual AI system that processes them. We get to choose between different modes of mouse operation like “Gesture Mode”, “Gesture &

Voice Mode” and “Eye Mode”. In “Gesture & Voice Mode” microphone works along with camera only when the hand movement is captured over the frames. With the help of Speech Recognition the mouse operations like click, double click, etc can be performed using voice command if camera fails to provide the hand operation output. “Eye Mode” also works with the help of OpenCV and MediaPipe, eye movement is tracked which works as cursor movement over the window. In Eye Mode, we used voice command as the mouse clicking, selecting and further operations. Blinking or winking the eye can be difficult to use as the blinking is an involuntary action which will make the program to execute unwanted operations.

3.1.7 Video Recording and Analysing.

The webcam will be used by the AI virtual mouse system to record each frame till the program is finished. The images are captured and converted to RGB to allow for frame-by-frame identification of the hand and eye[3][7]. For the voice control, right click, left click and double click commands are used for performing the mouse operation.[11]

3.1.8 Virtual Screen Matching.

This is used to move the hand coordinates between the web cam and the computer’s window to execute certain mouse functions, the AI virtual mouse technique enables a transformational method. After the recognition of the finger tips and hands we are notified of which fingers are capable of performing a cursor movement, a rectangle box may be generated on the computer screen showing the reference of the web cam. From there, we can see the cursor movements around the window[3][10].

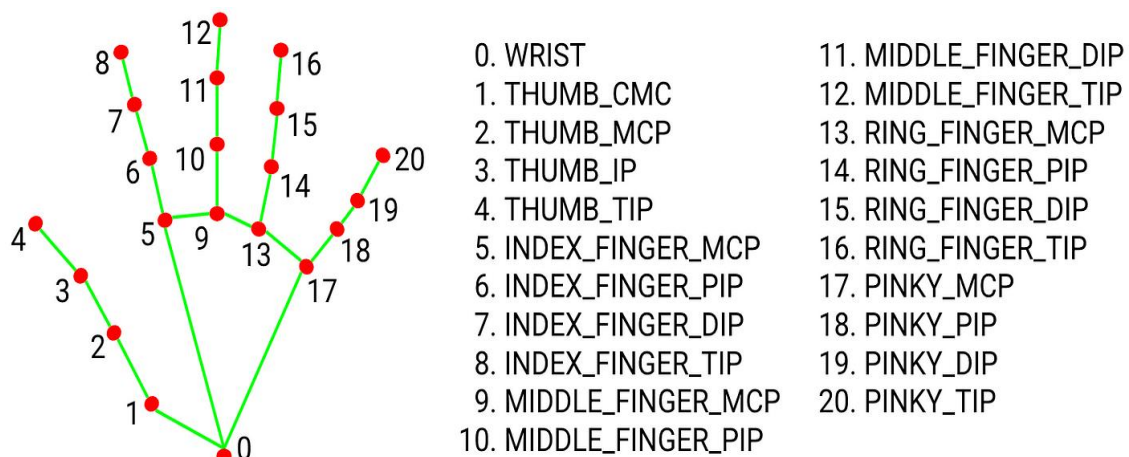


Fig 3.6: Co-ordinates in hand

3.1.9 Recognizing the Finger which is up and Performing Out the Appropriate Mouse Operation.

In order to transfer the hand coordinates between the webcam to the computer windows full screen for mouse operation, the AI virtual mouse technique enables a transformational mechanism.

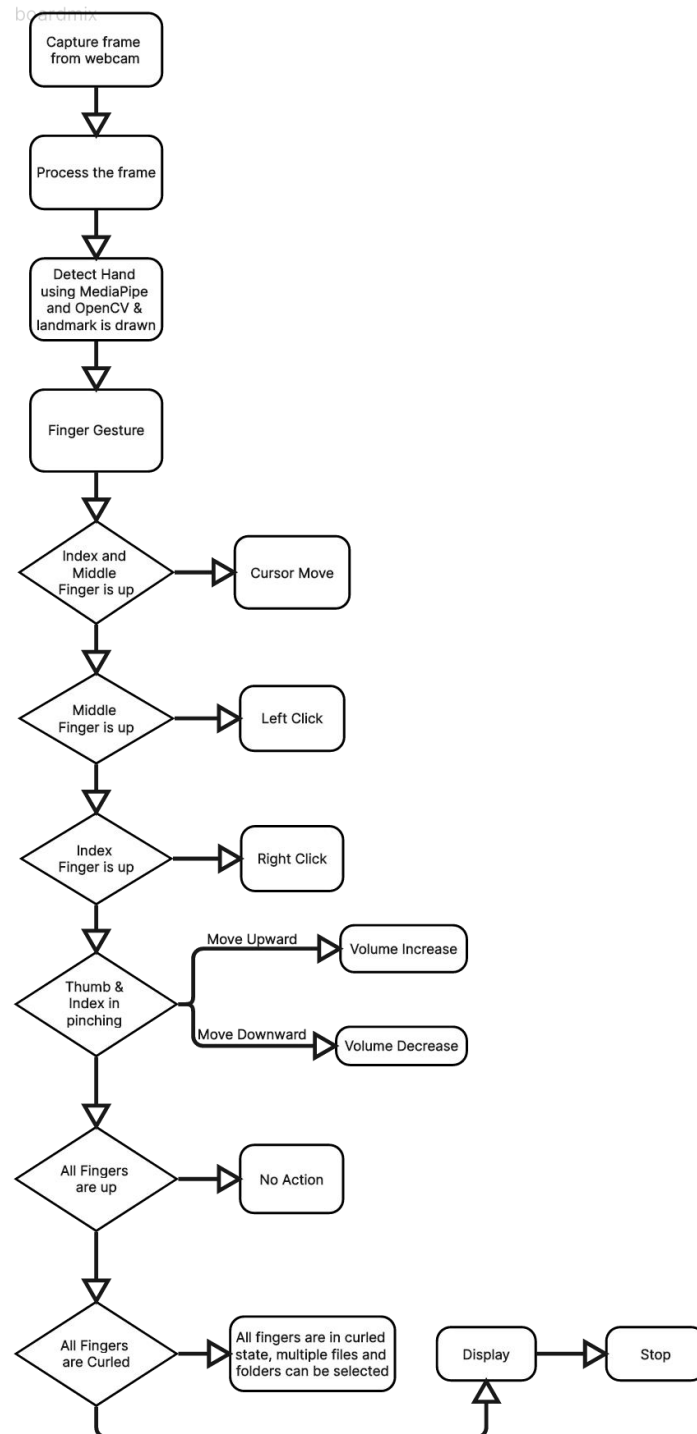


Fig 3.7: Mouse Functions

After the hands are detected and we are notified of which finger is capable of performing a certain mouse movement, a rectangular box is formed in reference to the computer window in the camera region. From that, we may move the mouse pointer around the window. Some of the gesture mouse actions are:

Cursor Movement - The midpoint of the index and middle fingertips is where the cursor is located. Cursor moves as the midpoint moves.

No Action (Neutral) - Stop execution of current gesture. All fingers are up.

Left Click - Middle finger up and index finger down.

Right Click - Index finger up and middle finger down.

Double Click - Index finger and middle finger is touched together.

Scrolling - Pinch gesture moving vertically and horizontally for scrolling according to layout.

Item selection, Drag & Drop - All fingers are curled over the file for selecting and dragging, then for dropping use neutral action.

Volume & Brightness Control - Pinch gesture over the windows moving vertically and horizontally to change brightness and volume of the PC.

3.1.10 Recognizing the Commands used for Different Modes of Mouse Operation.

In Gesture & Voice Mode the commands used for mouse events were “Left Click”, “Right Click”, “Double Click”, “Scroll Up”, “Scroll Down”, “Increase Volume”, “Decrease Volume”, “Increase Brightness” and “Decrease Brightness”. The mouse operations is same as the named command so there is no need of confusion when the mouse performs the task.

In Eye Mode, the commands were “Select”- (Left Click), “Right Click” and other commands are also same as in Gesture & Voice Mode. For Double Click, the user needs to say “Select” twice during that listening phase so the double click operation is performed.

3.2 System Architecture

3.2.1 GUI

Created a GUI using CustomTkinter to provide a window for accessing the different modes of mouse controlling[11]. Used presentation layer (GUI Toolkit) the visual elements that is

seen on the screen, like buttons, text labels, and the mode icons. CtkButton, CtkFrame, CtkLabel, CtkFont, and Tkinter PhotoImage is used.

Components used are:

Input Handling: This layer detects and processes user input. Depending on the chosen mode, it could involve:

Gesture Recognition: A software module might analyze camera to interpret gestures.

Speech Recognition: A speech recognition engine might convert spoken commands to text.

Eye Tracking: An eye tracking module could determine where the user is looking on the screen.

Control Logic: This layer interprets the user's chosen mode and input. Based on the selection (Gesture, Gesture & Voice, Eye), it activates the appropriate sub-modules for further processing.

Application Logic: This layer interacts with the underlying functionality of the system. It might send commands to control a device, access data, or perform actions based on user input processed in the previous layer.

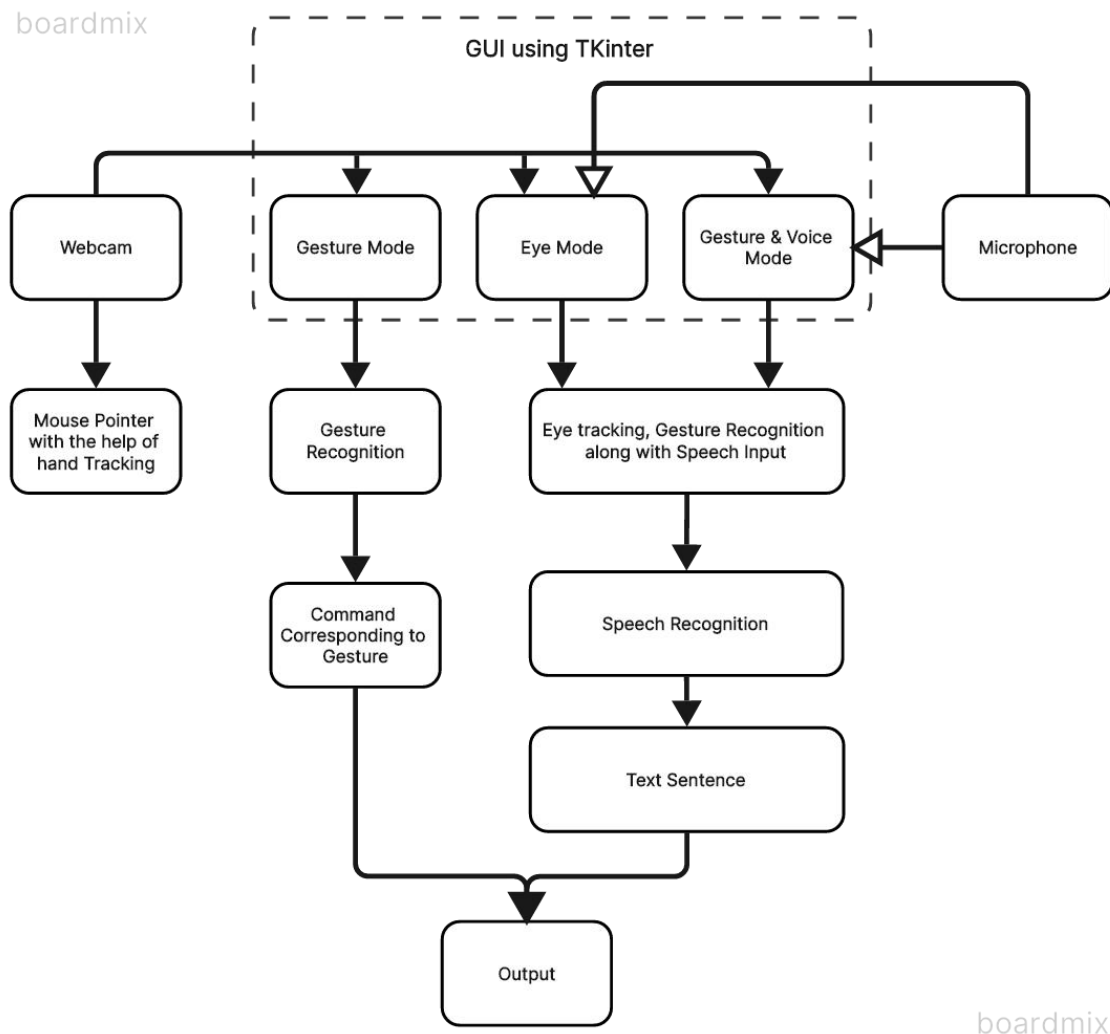


Fig 3.8: Architecture of accessing Different Modes of Operation

3.2.2 Gesture Mode

A gesture controlled virtual mouse using AI system typically works by following these steps: The AI system used in gesture controlled virtual mouse systems is typically based on machine learning. Machine learning algorithms are trained on a dataset of images and gestures, and they learn to identify patterns in the data. This allows the algorithms to identify new gestures and hand positions accurately.

Here is a more detailed overview of the different components of a gesture controlled virtual mouse using AI system:

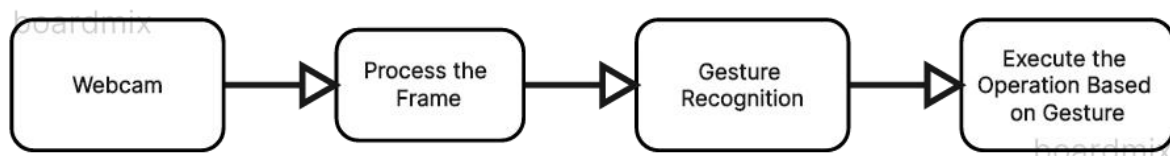


Fig 3.9: Flow Chart of Gesture Mode

3.2.2.1 Image Capture

The image capturer in an AI virtual mouse system is a critical component, as it is responsible for capturing high-quality images of the user's hand and eye gestures. The quality of the captured images directly impacts the accuracy of the system's hand detection and gesture recognition algorithms[3][11].

There are a number of different types of image capturers that can be used in AI virtual mouse systems. The most common type is a webcam, which is a low-cost and easy-to-use option. However, webcams can be susceptible to noise and occlusion, which can impact the accuracy of the system.

Another type of image capturer that can be used in AI virtual mouse systems is a depth camera, such as a Microsoft Kinect or a RealSense camera. Depth cameras provide additional information about the scene, such as the distance to each object in the image[2]. This information can be used to improve the accuracy of the system's hand detection and gesture recognition algorithms.

Finally, some AI virtual mouse systems use multiple image capturers to capture images of the user's hand from different angles. This can help to improve the accuracy of the system in challenging conditions, such as when the user's hand is partially occluded.

Here are some key factors to consider when choosing an image capturer for an AI virtual mouse system:

- **Resolution:** The resolution of the image capturer should be high enough to capture clear and detailed images of the user's hand and gestures.
- **Framerate:** The framerate of the image capturer should be high enough to capture smooth and fluid video of the user's hand and gestures.
- **Field of view:** The field of view of the image capturer should be wide enough to capture the user's entire hand and arm.
- **Noise immunity:** The image capturer should be resistant to noise, such as ambient light and electrical interference.
- **Occlusion robustness:** The image capturer should be able to handle occlusion, such as when the user's hand is partially blocked by another object.

3.2.2.2 Hand Detection

Hand detection is the process of identifying the hand in an image and tracking its position. It is a critical component of AI virtual mouse systems, as it allows the system to determine where the user's hand is and what gestures they are performing[9].

There are a number of different hand detection algorithms that can be used in AI virtual mouse systems. Some of the most common algorithms include:

- **Skin colour detection:** This algorithm identifies the hand by detecting the skin pixels in the image.
- **Edge detection:** This algorithm identifies the hand by detecting the edges of the hand in the image.
- **Contour detection:** This algorithm identifies the hand by detecting the contours of the hand in the image.
- **Deep learning:** This algorithm uses deep learning to identify the hand in the image.

Deep learning-based hand detection algorithms have become increasingly popular in recent years, as they have been shown to achieve high accuracy. However, deep learning algorithms can be computationally expensive, which can make them impractical for real-time applications.

Once the hand has been detected, the system can track its position by following the movement of the hand over time. This can be done using a variety of different tracking algorithms, such as Kalman filtering or optical flow.

Hand detection and tracking algorithms are essential for AI virtual mouse systems. By accurately detecting and tracking the user's hand, these algorithms allow the system to provide a smooth and responsive user experience.

Here are some key factors to consider when choosing a hand detection and tracking algorithm for an AI virtual mouse system:

- **Accuracy:** The algorithm should be able to accurately detect and track the user's hand in a variety of conditions, such as different lighting conditions and backgrounds.
- **Speed:** The algorithm should be fast enough to run in real time.
- **Robustness:** The algorithm should be robust to noise and occlusion.
- **Computational complexity:** The algorithm should be computationally efficient, so that it can be run on a variety of devices.

3.2.2.3 Gesture Recognition

Gesture recognition is the process of identifying the gesture that a user is performing. It is a critical component of AI virtual mouse systems, as it allows the system to determine what action the user wants to perform.

There are a number of different gesture recognition algorithms that can be used in AI virtual mouse systems. Some of the most common algorithms include:

- **Template matching:** This algorithm identifies gestures by matching them to a set of pre-defined templates.
- **Hidden Markov models (HMMs):** HMMs are a type of statistical model that can be used to recognize sequential patterns, such as hand gestures[4].
- **Deep learning:** Deep learning-based gesture recognition algorithms have become increasingly popular in recent years, as they have been shown to achieve high accuracy.

Deep learning-based gesture recognition algorithms typically work by first extracting features from the hand image. These features can include the position of the hand and fingers, the orientation of the hand, and the motion of the hand over time. The features are then fed into a deep learning model, which is trained to classify the gesture.

Gesture recognition algorithms in AI virtual mouse systems are typically trained on a dataset of hand images and gestures. The dataset should include a variety of different gestures, as well as images of the hand in different lighting conditions and backgrounds.

Once the gesture recognition algorithm is trained, it can be used to identify new gestures in real time. This allows the AI virtual mouse system to provide a smooth and responsive user experience.

Here are some key factors to consider when choosing a gesture recognition algorithm for an AI virtual mouse system:

- **Accuracy:** The algorithm should be able to accurately recognize gestures in a variety of conditions, such as different lighting conditions and backgrounds.
- **Speed:** The algorithm should be fast enough to run in real time.
- **Robustness:** The algorithm should be robust to noise and occlusion.
- **Computational complexity:** The algorithm should be computationally efficient, so that it can be run on a variety of devices.

3.2.2.4 Mouse Cursor Control

Mouse cursor control in AI virtual mouse systems is the process of using the output of the hand detection and gesture recognition algorithms to control the position and movement of the mouse cursor on the screen[3][10][11].

There are a number of different ways to implement mouse cursor control in AI virtual mouse systems. One common approach is to map the position of the hand and fingers to the position of the mouse cursor on the screen. For example, the system could map the position of the index finger to the position of the mouse cursor.

Another approach to mouse cursor control is to use gestures to perform specific mouse actions, such as clicking, dragging, and scrolling. For example, the system could use a gesture to indicate a click and then use the position of the hand to indicate the location of the click.

The specific approach to mouse cursor control that is used in an AI virtual mouse system will depend on the specific design of the system. However, all AI virtual mouse systems must be able to accurately and reliably control the position and movement of the mouse cursor in order to provide a useful and productive experience for the user.

Here are some key factors to consider when designing a mouse cursor control algorithm for an AI virtual mouse system:

- **Accuracy:** The algorithm should be able to accurately map the position of the hand and fingers to the position of the mouse cursor on the screen.
- **Smoothness:** The algorithm should be able to control the movement of the mouse cursor in a smooth and fluid manner.
- **Responsiveness:** The algorithm should be able to respond to changes in the position of the hand and fingers in real time.
- **Robustness:** The algorithm should be robust to noise and occlusion.
- **Usability:** The algorithm should be easy to use and should not require the user to perform complex gestures.

Overall, mouse cursor control is a critical component of AI virtual mouse systems. By designing a well-implemented mouse cursor control algorithm, system designers can improve the accuracy, smoothness, responsiveness, robustness, and usability of their systems.

Here are some examples of advanced mouse cursor control algorithms that can be used in AI virtual mouse systems:

Virtual screen matching: This algorithm maps the hand coordinates from the webcam screen to the computer window screen, allowing the user to control the mouse cursor throughout the window using a rectangular box.

Gesture-based mouse control: This algorithm allows the user to control the mouse cursor using gestures, such as moving the hand, clicking, and scrolling.

Air mouse control: This algorithm allows the user to control the mouse cursor by moving their hand in the air.

3.2.3 Gesture & Voice Mode

In this mode, hand gestures are all same as in Gesture Mode additionally a Speech Recognition is implemented to work as a voice operated mouse operations[11]. Speech Recognition process works as:

Audio Capture: The user speaks a voice command through the microphone.

Audio Preprocessing: The raw audio might undergo noise reduction, filtering, or other processing to improve speech clarity.

Feature Extraction: Specific features like Mel-Frequency Cepstral Coefficients (MFCCs) are extracted from the preprocessed audio. These features capture the characteristics of the speech signal.

Acoustic Model: A trained acoustic model analyzes the extracted features. This model is typically a large neural network that has been trained on a massive dataset of speech and corresponding text. The model calculates the probability of each feature sequence corresponding to specific phonemes (basic units of sound).

Language Model: A language model predicts the most likely sequence of words based on the probabilities from the acoustic model and its knowledge of grammar and vocabulary.

Decoder: The decoder combines the outputs from the acoustic model and the language model to generate the most likely sentence that corresponds to the spoken command.

Mapping Voice Commands to Clicks:

Voice Command Dictionary: A predefined dictionary maps specific spoken phrases or keywords to corresponding virtual mouse actions. For example, "left click" or "click" could be mapped to a left click action.

Speech Recognition Output: The speech recognition engine outputs the recognized text (sentence).

Command Matching: The system checks the recognized text against the voice command dictionary.

Action Translation: If a match is found, the corresponding virtual mouse action (left click in this case) is identified.

3.2.4 Eye Mode

The face is captured using the OpenCV library and then sent to the MediaPipe library. The Python library known as OpenCV (Open Source Computer Vision Library) is an open source machine learning and computer vision software library. The purpose of OpenCV is to expedite the integration of machine perception into commercial goods by offering a common foundation for computer vision applications[7]. It continually records video using the laptop camera by utilizing the OpenCV library's VideoCapture function. The collected frames are fed into the MediaPipe machine learning framework, which uses them to identify the various

eye elements and forecast the face. MediaPipe uses the SVM algorithm to identify faces. This exercise can be used to identify faces inside a frame and identify distinguishing characteristics on faces. A machine learning (ML) model that can process either a single image or an ongoing stream of images is used for this task. Six face landmarks—the eyes, ears, nose, and mouth—as well as facial locations are output by the job. MediaPipe recognizes and marks various facial features. To mark landmarks on the face, we use the MediaPipe library's FaceMesh method.

The camera locates the eye's pupil and with the help of pupil coordinates the cursor is moved. In this mode the user can't use blinking as the mouse clicking operation because the blinking is an involuntary action so the user find it difficult to use them. Instead user can use voice command for clicking actions. The voice will work after you blink once over the camera to make the system listen to the voice, recognize and execute the command.

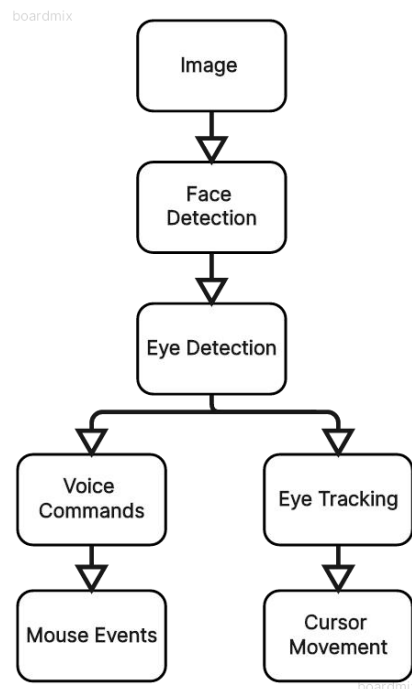


Fig 3.10: Flow Chart of Eye Mode

CHAPTER 4

RESULT AND DISCUSSION

4.1 Experimental Setup

This Virtual Mouse Control System is designed to give ease of access to the user without using the conventional mouse. This also helps disabled people to use different modes of mouse operation according to their convenience. System provides gesture, voice and eye to use the mouse. This system can be used on any platform. It works on HDD but a little bit of lag can be seen. We recommend a high speed processor, internet connection and SSD for optimal performance.

4.2 Screenshots

4.2.1 Hand Tracking

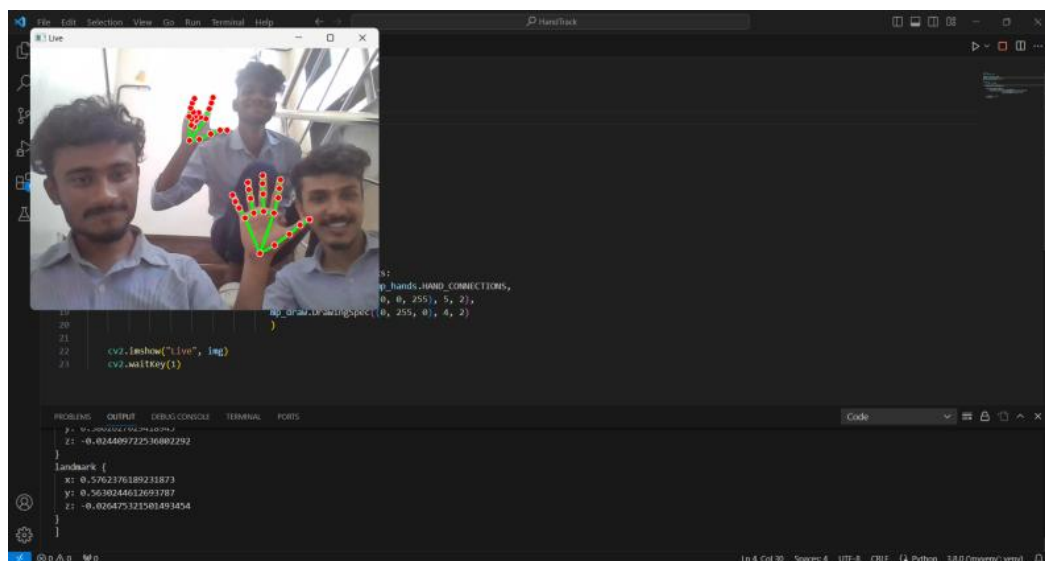


Fig 4.1: Hand Tracking

Implemented Hand Tracking through hand landmarks with the help of OpenCV and MediaPipe. When hand moves over the webcam, the landmark position can be seen changing its values. With this hand landmark movement, different mouse functions can be implemented.

4.2.2 GUI

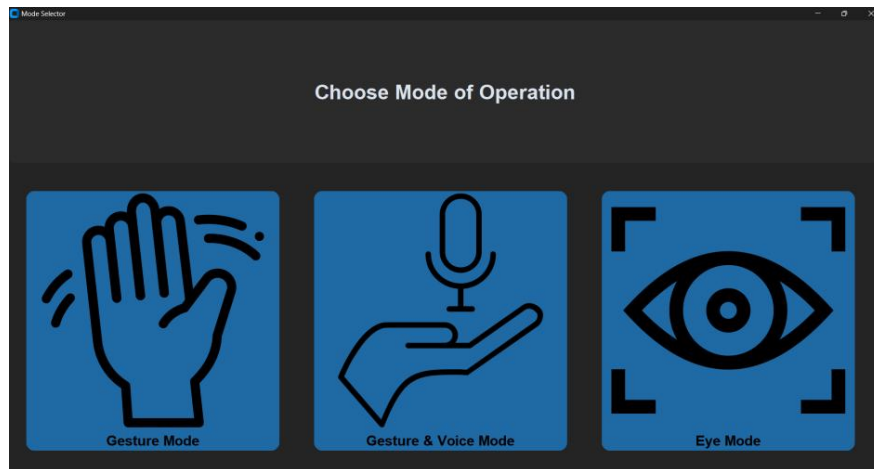


Fig 4.2: Mode Selector

This is the Mode Selector which is used to select the modes of mouse operation created using TKinter (GUI).

4.2.3 Gesture Mode

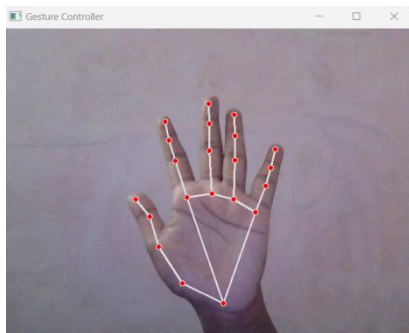


Fig 4.3: Neutral Action



Fig 4.4: Cursor Move

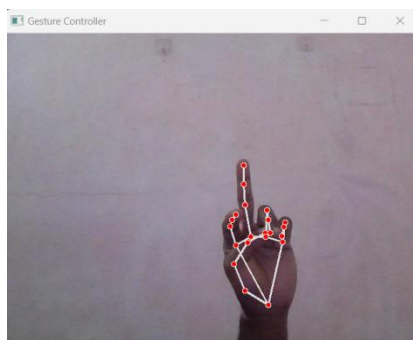


Fig 4.5: Left Click

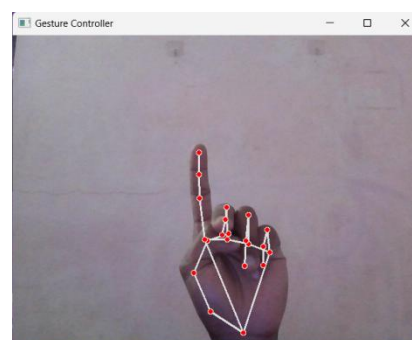


Fig 4.6: Right Click

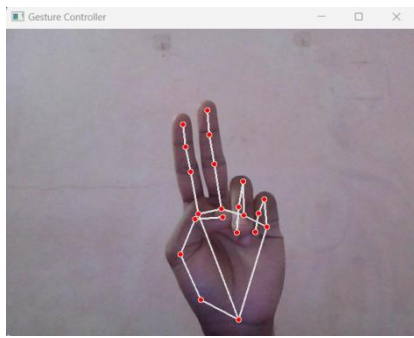


Fig 4.7: Double Click

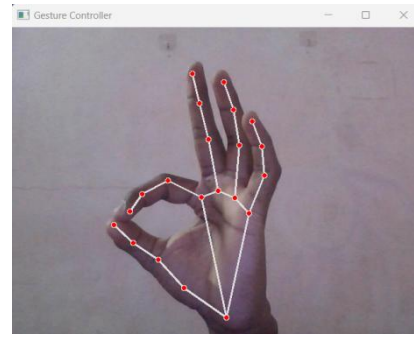


Fig 4.8: Scrolling, Brightness and Volume Control

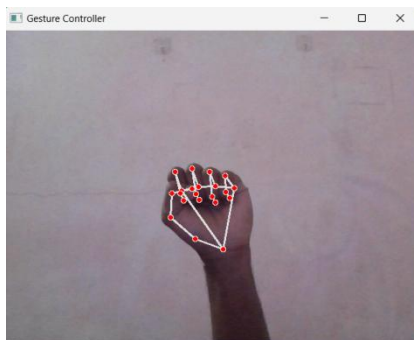


Fig 4.9: Drag and Drop

The above figures are the basic mouse functions performed using hand gesture via webcam. This is used in both Gesture Mode and Gesture & Voice Mode. In Gesture & Voice Mode the user can use either hand gesture or voice command to perform mouse functions.

4.2.4 Gesture & Voice Mode

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SEARCH ERROR
Click here to ask Blackbox to help you code faster
[Running] python -u "c:\Users\...\Desktop\PROJECT\Final.py"
C:\Users\...\AppData\Local\Programs\Python\Python38\lib\site-package
image is not CTKImage but <class 'tkinter.PhotoImage'>. Image can not
warnings.warn(f"{type(self).__name__} Warning: Given image is not C
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
Listening for voice commands...

Listening for voice commands...
Recognized command: left click

```

Fig 4.10: Left Click Command Execution

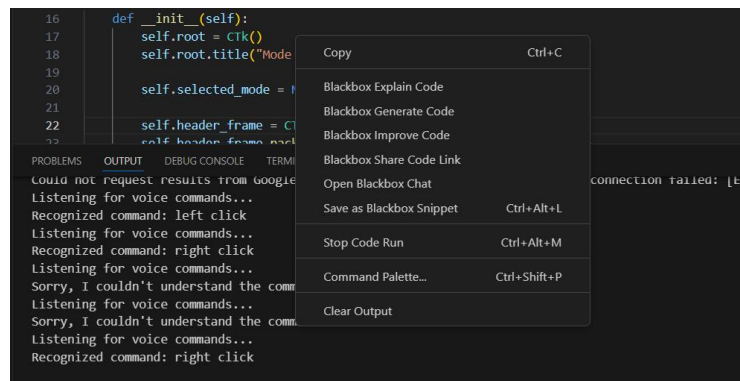


Fig 4.11: Right Click Command Execution

These are voice commands and its execution performed on Gesture & Voice Mode and Eye Mode. In Gesture & Voice Mode, speech recognition won't work every time that is microphone takes input only after the hand is captured over the webcam and it starts to listen then recognize the command. In Eye Mode, blinking initiates the microphone usage.

4.2.5 Eye Mode

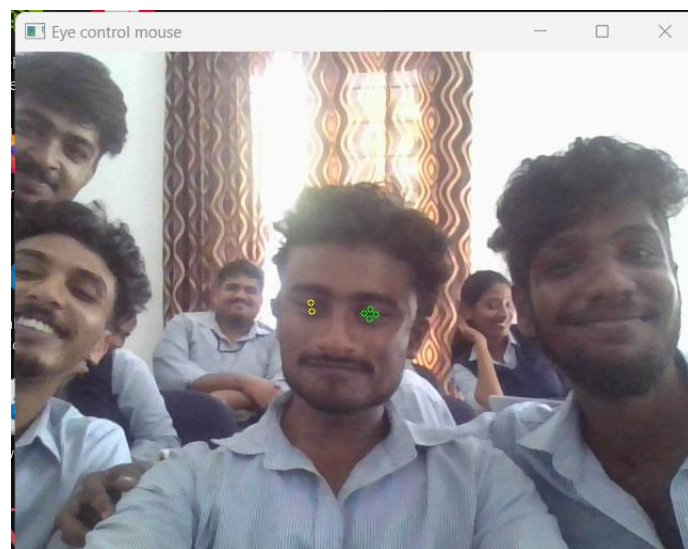


Fig 4.12: Cursor Movement using Eye

The Eye landmark is used as the placement for cursor and its movement over the window.

CHAPTER 5

CONCLUSION

The development of effective human-machine interaction has become significantly impacted by the hand gesture detection, voice assistant systems and eye detection. Wide- ranging applications in the technology sector are promised by implementation employing hand gesture recognition. The MediaPipe, a machine learning framework, has a significant impact on the creation of this application that uses hand gesture and eye recognition.

Advanced Virtual Mouse Gesture and Voice Control System is a promising new technology with the potential to revolutionize the way we interact with computers and other devices. It offers a number of advantages over traditional mouse control methods, including hygiene, accessibility, immersive, flexibility, and ergonomics.

Compared to conventional mouse movements, this application can be quicker and more natural, which helps users finish tasks more quickly and with less mental strain. Because ML can recognize intricate gestures, we can assign new task or action to that kind of gestures. Using gestures to switch between windows, apps, and desktops can simplify workflows and lessen the difficulty of switching between contexts. This also prevent injuries from repetitive strain and encourage improved hand health.

CHAPTER 5

FUTURE SCOPE

The future scope for Advanced Virtual Mouse Gesture and Voice Control System is very promising. As the technology continues to develop and become more affordable, we can expect to see it being used in even more applications and by more people.

Here are some specific examples of how Gesture Controlled Virtual Mouse using AI could be used in the future:

- In education, this could be used to create immersive and interactive learning experiences for students. For example, students could use gestures to control virtual robots or explore virtual worlds.
- In healthcare, this could be used to help people with disabilities interact with computers and other devices. For example, people with paralysis could use gestures to control their wheelchairs or to operate prosthetics.
- In the workplace, this could be used to improve productivity and efficiency. For example, workers could use gestures to control their computers and other devices without having to take their hands off their work.
- In the home, this could be used to control smart home devices and appliances. For example, people could use gestures to turn on the lights, adjust the thermostat, or change the channel on the TV.
- In gaming and entertainment, this could be used to create more immersive and engaging experiences for users. For example, gamers could use gestures to control their characters in video games or to interact with virtual objects in augmented reality and virtual reality.

In addition to these specific applications, Advanced Virtual Mouse Gesture and Voice Control System has the potential to revolutionize the way we interact with computers and other devices in general. By making it possible to control computers and other devices using natural gestures, virtual mouse control system can make our lives easier, more convenient, and more productive.

Overall, the future scope for Advanced Virtual Mouse Gesture and Voice Control System is very bright. As the technology continues to develop and become more widely adopted, we can expect to see Gesture Controlled Virtual Mouse using AI having a significant impact on our lives.

REFERENCES

- [1] D. L. Quam, "Gesture recognition with a Data Glove," IEEE Conference on Aerospace and Electronics, vol. 2, pp. 755–760, 1990.
- [2] D.H. Liou, D. Lee, and C.C. Hsieh, "A real time hand gesture recognition system using motion history image," in Proceedings of the 2010 2nd International Conference on Signal Processing Systems, July 2010.
- [3] V. V. Reddy, T. Dhyanchand, G. V. Krishna and S. Maheshwaram, "Virtual Mouse Control Using Colored Finger Tips and Hand Gesture Recognition," 2020 IEEE-HYDCON, 2020, pp. 1-5, DOI: 10.1109/HYDCON48903.2020.9242677.
- [4] L. R. Rabiner B. H. Juang "An Introduction to Hidden Markov Models" IEEE ASSP Magazine (Volume: 3, January 1986)
- [5] Akshaya U Kulkarni, Amit M Potdar "RADAR based Object Detector using Ultrasonic Sensor" 2019 1st International Conference on Advances in Information Technology (ICAIT) 10 February 2020
- [6] M.K. Bhuyan, D. Ghosh and P.K. Bora "Co-articulation Detection in Hand Gestures" TENCON 2005 - 2005 IEEE Region 10 Conference February 2007.
- [7] Mr. Dhanaraju, Dr. Sreenivas Mekala, A. Harsha Vardhan Rao, CH. Pavan Kumar, R. Lokesh "Human-Eye Controlled Virtual Mouse" International Journal for Research in Applied Science & Engineering Technology (IJRASET) ISSN: 2321-9653; IC Value: 45.98; SJ Impact Factor: 7.538 Volume 10 Issue VI June 2022
- [8] S. Shriram, B. Nagaraj , "Deep Learning-Based Real-Time AI Virtual Mouse" Volume 2021 ArticleID 8133076 <https://doi.org/10.1155/2021/8133076>
- [9] V. Bazarevsky and G. R. Fan Zhang. On-Device, MediaPipe for Real-Time Hand Tracking.
- [10] A. Haria, A. Subramanian, N. Asok kumar, S. Poddar, and J. S. Nayak "Hand gesture recognition for human compute interaction, "Procedia Computer Science, vol. 115, pp. 367–374, 2017.
- [11] Tata Jagannadha Swamy, M Nandini, Nandini B, Venkata Karthika K, V Laxmi Anvitha and Ch Sunitha "Voice and Gesture based Virtual Desktop Assistant for Physically Challenged People"